

Object-Oriented Programming

Contents

Object-Oriented Programming	3
1. Approach	3
2. Encapsulation.....	4
3. Abstraction	12
4. Inheritance	15
5. Combine Concepts	19
6. Polymorphism.....	20
References.....	20
Reference in Python	21
Reference in Java	21
Reference in C#	22
Reference in C++.....	22
Two Main Types of Polymorphism.....	22
7. Abstract Classes and Abstract Methods.....	27
Abstract Method:	27
8. Interfaces.....	29
9. When to use Abstract classes and when to use Interfaces	31
Use Abstract Classes When:.....	31
Use Interfaces When:	31
Rule of Thumb:	31
Summary Comparison Table.....	32
More example	32

Object-Oriented Programming

1. Approach

- Object-Oriented Programming (OOPs) is a programming approach that organizes code into **objects** and **classes** and makes it more structured and easier to manage.
- For example, people who have used **C programming language**, knows that we organize the code using **functions** and **procedures**.

- **Procedures:** It is essentially a **function that performs a specific task** but does **not return a value**. It can take inputs (**parameters**) even though it does not return a value.

```
void greetUser(char name[])
{
    printf("Hello, %s!\n", name);
}
```

- **Functions:** A **function** is a **block of code** that can take input (parameters), **perform some operation**, and **optionally return a value**.

```
int add(int a, int b)
{
    return a + b;
}
```

- Summary Table:

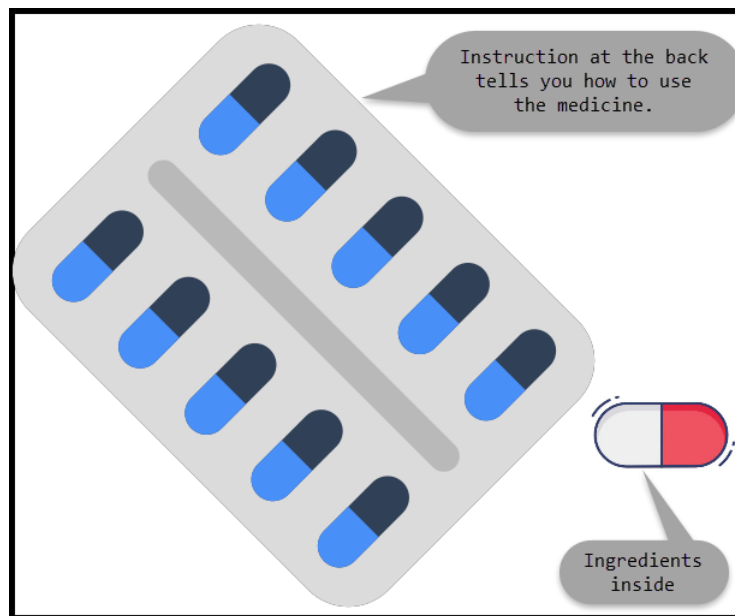
No.	Feature	C (Procedural)	OOP (e.g., C++, Java)
1	Approach	Function-based	Object-based
2	Data Handling	Globally accessible	Data hidden in objects
3	Reusability	Limited	High (via inheritance)
4	Security	Low	High
5	Modularity	Through functions	Through classes/objects
6	Flexibility	Less	More (supports overloading, etc.)

- We understood the 1 (Approach/Paradigm) in the above table.

2. Encapsulation

Note: When you are designing a class in an object-oriented language, following the principle of encapsulation is highly recommended and considered good practice.

- Imagine you have a medicine capsule. Inside the capsule are the actual ingredients (our data). You can't directly touch or change these **ingredients**. Instead, the capsule itself has instructions printed on it (our public interface or methods). These instructions tell you how to use the medicine – maybe "take one capsule with water" or "dissolve under the tongue."



- In programming, encapsulation is like that capsule:
 - **Private Data (The Ingredients Inside):**
 - This is the information or variables that an object holds.
 - We make this data "private" so that it can't be directly accessed or modified from outside the object.
 - This protects the data from accidental changes that could break things.
 - **Public Interface (The Instructions on the Capsule):**
 - These are the methods (functions that belong to the object) that allow you to interact with the object's data in a controlled way.
 - These methods define how you can access or modify the private data. They act as a gatekeeper.

- Why do we do this "data hiding" and "bundling"?
 - **Protection** (Like the **capsule** protecting the **medicine**): By making data **private**, we prevent external code from directly messing with the internal workings of our object. This makes our code more robust and less prone to errors. If someone tries to change the data directly, the programming language will usually prevent it.
 - **Control** (Following the **instructions ensures correct usage**): The **public methods** provide a controlled way to interact with the data. We can define rules and logic within these methods to ensure that the data is modified in a valid way. For example, a method to set someone's **age** might include a check to make sure the **age** isn't **negative**.
 - **Flexibility** and **Maintainability** (If the ingredients change, the instructions might stay the same): If we need to change how the data is stored internally, as long as we keep the public methods the same (the "instructions" remain consistent), the code that uses our object won't break. The internal implementation can change without affecting the external interface.
- Example:

```
#include <iostream>
#include <string>

using namespace std;

class Car {
private:
    // Private member variables
    string engine = "V6";
    int fuelLevel = 50;

public:
    void accelerate()
    {
        cout << "Car is accelerating." << endl;
        fuelLevel -= 1; // Modifying private data within a public method
    }
}
```

```

int getFuelLevel()
{
    return fuelLevel; // Providing controlled access to private data
}

// New public method to set the fuel level with validation
void setFuelLevel(int newFuelLevel)
{
    if (newFuelLevel >= 0 && newFuelLevel <= 100)
    {
        fuelLevel = newFuelLevel;
        cout << "Fuel level set to: " << fuelLevel << endl;
    }
    else
    {
        cout << "Invalid fuel level. "<<
            "Please enter a value between 0 and 100.\n";
    }
}
};

int main() {
    Car myCar;

    cout << "Initial fuel level: " << myCar.getFuelLevel() << endl;

    // Attempting to set an invalid fuel level
    myCar.setFuelLevel(-100);
    cout << "Current fuel level after attempting "
        <<"to set invalid value : " << myCar.getFuelLevel() << endl;

    // Setting a valid fuel level
    myCar.setFuelLevel(80);
    cout << "Current fuel level after setting "
        <<"a valid value : " << myCar.getFuelLevel() << endl;

    myCar.accelerate();
    cout << "Current fuel level after acceleration: "
        << myCar.getFuelLevel() << endl;

    return 0;
}

```

- Some other examples:
 - `class: Student`
 - `private: name, rollNumber, marks`
 - `public: setName(), getName(), setMarks(), getMarks()`

```
class Student:
    def __init__(self, name, roll_number):
        # Private attributes (With double underscore)
        self.__name = name
        self.__roll_number = roll_number
        self.__marks = {} # Dictionary to store marks for different subjects

    # Public methods to set and get the name
    def set_name(self, new_name):
        if isinstance(new_name, str) and new_name.strip():
            self.__name = new_name
        else:
            print("Invalid name format.")

    def get_name(self):
        return self.__name

    # Public methods to set and get marks
    def set_marks(self, subject, mark):
        if isinstance(subject, str) and subject.strip() \
            and isinstance(mark, (int, float)):
            if 0 <= mark <= 100: # Assuming marks are out of 100
                self.__marks[subject] = mark
            else:
                print(f"Invalid mark for {subject}. \
                    Marks should be between 0 and 100.")
        else:
            print("Invalid subject or mark format.")

    def get_marks(self, subject):
        if subject in self.__marks:
            return self.__marks[subject]
        else:
            return f"Marks for {subject} not found."

    def get_all_marks(self):
        return self.__marks
```

```

# Creating a Student object
student1 = Student("Alice", "S101")

# Accessing and modifying data using public methods
student1.set_name("Alice Smith")
print(f"Name: {student1.get_name()}")

student1.set_marks("Math", 95)
student1.set_marks("Science", 88)
print(f"Marks in Math: {student1.get_marks('Math')}")
print(f"All marks: {student1.get_all_marks()}")

# Attempting to set invalid marks
student1.set_marks("History", 110)
student1.set_marks("English", -5)

# In Python, name mangling provides a degree of privacy, but it's not strict.
# You can still technically access 'private' attributes, but it's discouraged.
# Let's see how Python name mangles the attribute name:
# print(student1._Student__name) # This would work,
# but it's not the intended way
# The intended way is to use the public interface (the methods).

```

- **class:** BankAccount
- **private:** balance, accountNumber
- **public:** deposit(), withdraw(), checkBalance()

```

using System;

public class BankAccount
{
    // Private member variables
    private decimal balance;
    private string accountNumber;

    // Constructor to initialize the BankAccount object
    public BankAccount(string accountNumber, decimal initialBalance)
    {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }
}

```



```

// Public method to deposit funds
public void Deposit(decimal amount)
{
    if (amount > 0)
    {
        balance += amount;
        Console.WriteLine($"Deposited ${amount}. New balance: ${balance}");
    }
    else
    {
        Console.WriteLine("Deposit amount must be positive.");
    }
}

// Public method to withdraw funds
public void Withdraw(decimal amount)
{
    if (amount > 0)
    {
        if (balance >= amount)
        {
            balance -= amount;
            Console.WriteLine($"Withdrew ${amount}. New balance: ${balance}");
        }
        else
        {
            Console.WriteLine("Insufficient balance.");
        }
    }
    else
    {
        Console.WriteLine("Withdrawal amount must be positive.");
    }
}

// Public method to check the current balance
public decimal CheckBalance()
{
    return balance;
}

// Public method to get the account number (read-only access)
public string GetAccountNumber()
{

```

```

        return accountNumber;
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        // Create a BankAccount object
        BankAccount myAccount = new BankAccount("1234567890", 1000.00m);

        Console.WriteLine($"Account Number: {myAccount.GetAccountNumber()}");
        Console.WriteLine($"Initial Balance: ${myAccount.CheckBalance()}");

        // Perform some transactions using public methods
        myAccount.Deposit(500.00m);
        myAccount.Withdraw(200.00m);
        myAccount.Withdraw(1500.00m); // Attempt to withdraw more than the balance

        Console.WriteLine($"Current Balance: ${myAccount.CheckBalance()}");

        // Attempting to directly access private members (compile-time error)
        // myAccount.balance = -100.00m;
        // Console.WriteLine(myAccount.balance);
    }
}

```

- Class: TemperatureSensor
- Private: celsius
- Public: setCelsius(), getFahrenheit()

```

public class TemperatureSensor {
    private double celsius; // Private member variable to store temperature

    // Public method to set the temperature in Celsius
    public void setCelsius(double celsius) {
        this.celsius = celsius;
    }

    // Public method to get the temperature in Fahrenheit
    public double getFahrenheit() {
        // Conversion formula: Fahrenheit = (Celsius * 9/5) + 32
        return (this.celsius * 9.0 / 5.0) + 32;
    }
}

```

```

}

// Optional: Public method to get the temperature in Celsius (if needed)
public double getCelsius() {
    return this.celsius;
}

public static void main(String[] args) {
    TemperatureSensor sensor = new TemperatureSensor();

    // Setting the temperature using the public method
    sensor.setCelsius(25.0);
    System.out.println("Temperature in Celsius: " + sensor.getCelsius());

    // Getting the temperature in Fahrenheit using the public method
    double fahrenheit = sensor.getFahrenheit();
    System.out.println("Temperature in Fahrenheit: " + fahrenheit);

    // Attempting to directly access the private variable (compile-time error)
    // sensor.celsius = 30.0; // This line would cause an error

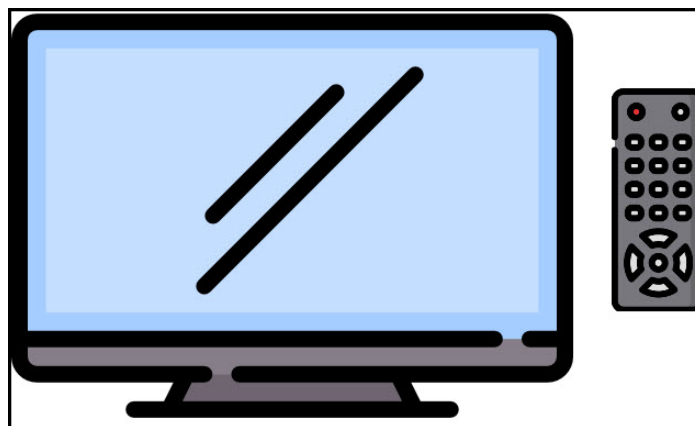
    // You can only change the Celsius value through the
    public interface (setCelsius method)
    sensor.setCelsius(35.5);
    System.out.println("New Temperature in Celsius: " + sensor.getCelsius());
    System.out.println("New Temperature in Fahrenheit: " + sensor.getFahrenheit());
}
}

```

3. Abstraction

Note: When you are designing a class in an object-oriented language, focus on what an object does rather than how it does.

- Imagine you're using a TV remote control.
 - **What you see (the abstraction):** Buttons for power, volume up/down, channel up/down, mute, etc. Each button has a clear function. You press the volume up button, and the volume goes up. You press the channel down button, and the channel changes.
 - **What's hidden (the internal details):** Inside the remote and the TV are complex electronic circuits, microprocessors, and wireless communication protocols working together. You don't need to understand how these intricate details function to use the remote effectively. You just care that pressing the "volume up" button achieves the desired outcome.



- In programming, abstraction is similar:
 - It's about showing only the essential information and functionality to the user (**to the developer using your class**) and hiding the complex implementation details. You focus on **what an object does** rather than **how it accomplishes it**.
- Why is Abstraction Important in Programming?
 - **Simplifies Complexity:** It allows developers to work with complex systems without getting bogged down in the low-level details. This makes the code easier to understand, use, and manage.

- **Reduces Code Duplication:** By abstracting common functionalities into reusable components (like methods or classes), you avoid writing the same code multiple times.
- **Increases Maintainability:** If the internal implementation of a feature changes, as long as the external interface (the methods and properties you expose) remains the same, the code that uses your abstraction won't break. You can modify the "**how**" without affecting the "**what**."
- Example: Design and implement a software system to manage the shopping cart functionality for an e-commerce platform. This system should allow users to
 - Add items to their cart
 - Apply discount coupons
 - Generate a final bill reflecting the items, quantities, prices, and any applied discounts.

```
// Supports:
// 1. Adding items to cart
// 2. Applying coupons
// 3. Generating final bills.
class BillingSystem {
public:
    void addItem(const std::string& itemName, double price);
    void applyCoupon(const std::string& code);
    void generateFinalBill();

private:
    std::vector<std::pair<std::string, double>> items;
    std::string couponCode;
    void PrintBill(double total, double discount, double coupon,
        double tax, double finalAmount);
    double calculateSubtotal();
    double calculateDiscount(double total);
    double applyCouponDiscount(double amountAfterDiscount);
    double calculateTax(double afterCoupon);
};
```

- Focus on "What" the System Does, Not "How":
 - The **public** methods (addItem, applyCoupon, generateFinalBill) represent the **essential actions** that the user of my **class** can perform with

the billing system. They tell the user of my `class` *what* the system can do: add items, apply discounts, and produce a bill.

- As a user of the `BillingSystem class` (like in the `main()` function which is given below), he/she calls these methods without needing to know the steps involved in storing the items, checking coupon validity, calculating discounts, or formatting the bill.

```
int main(void)
{
    BillingSystem bill;

    bill.addItem("Laptop", 3000);
    bill.addItem("Mouse", 50);
    bill.addItem("Headphones", 150);

    bill.applyCoupon("SAVE100");

    bill.generateFinalBill();

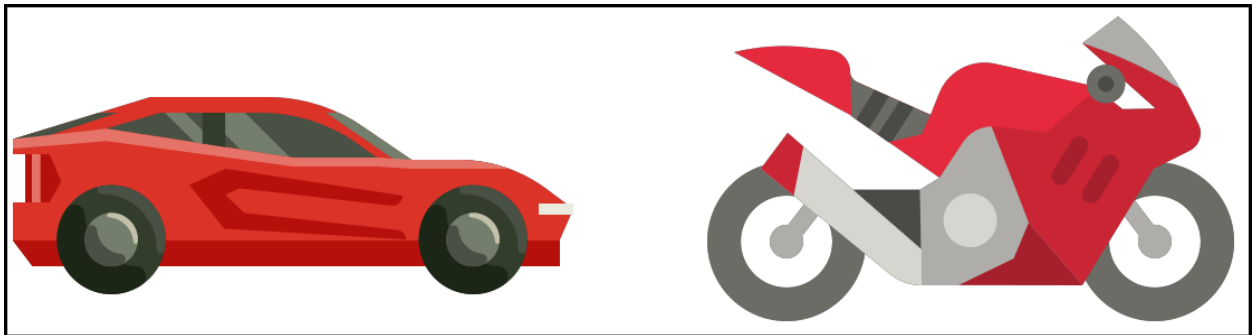
    return 0;
}
```

- Hiding Internal Complexity:
 - The `private` members (`items`, `couponCode`, `PrintBill`, `calculateSubtotal`, `calculateDiscount`, `applyCouponDiscount`, `calculateTax`) represent the internal mechanisms and data structures used by the `BillingSystem` to perform its tasks.
 - These `private` details are hidden from the outside. The `main` function doesn't directly interact with the `items` vector/list or the individual calculation methods. **This hiding of complexity is a key aspect of abstraction.** You don't need to know how the items are stored internally (e.g., using a vector of pairs) to add an item. You just use the `addItem` method.
- In essence, the `BillingSystem class` provides an abstraction over the complexities of calculating a bill. It exposes a clean and understandable interface (`addItem`, `applyCoupon`, `generateFinalBill`) while hiding the underlying implementation details (how items are stored, how discounts and taxes are calculated).

4. Inheritance

Note: When you are designing a class in an object-oriented language, establish "is-a" relationships **to reuse existing code** and build upon the common characteristics of more general concepts.

- Let's imagine you are developing a racing Game:
 - You have a blueprint for a general **Vehicle**. This blueprint includes common features that most vehicles have, like:
 - `startEngine()`
 - `stopEngine()`
 - `accelerate()`
 - `brake()`
- Now, you want to create blueprints for more specific types of vehicles, like a **Car** and a **Motorcycle**.



- Inheritance is like saying, "Hey, **Car** and **Motorcycle**, you are both types of **Vehicles**, so you **automatically get all the features from the Vehicle blueprint!**"
- Instead of writing the `startEngine()`, `stopEngine()`, `accelerate()`, and `brake()` functions again from scratch for both **Car** and **Motorcycle**, they inherit these functionalities from the **Vehicle** blueprint.
- Think of it as a family tree:
 - **Vehicle** is the parent class (or superclass, or base class).
 - **Car** and **Motorcycle** are the child classes (or subclasses, or derived classes).
- The children **inherit** characteristics (in our case, functionalities or methods) from their parent.

- But wait, **Car** and **Motorcycle** are also different!
 - A **Car** might have a **openDoors()** method and a **turnOnRadio()** method, which a **Motorcycle** doesn't have.
 - A **Motorcycle** might have a **lean()** method, which a **Car** doesn't have.
- Inheritance allows child classes to:
 - **Reuse code:** They automatically get the properties and methods of the parent **class**.
 - **Add new features:** Child **classes** can have their own unique properties and methods that are specific to their type. **Car** gets **openDoors()**, and **Motorcycle** gets **lean()**.
 - **Override or modify existing features:** A child **class** can redefine a method it inherited from the parent **class** to behave in a way that's specific to it. For example, the **accelerate()** method in a **SportsCar** might behave differently (more aggressively) than the **accelerate()** method in a regular **Car**.
- "Is-a" Relationship:
 - Inheritance establishes an "is-a" relationship between the **classes**.
 - A **Car** **is a** **Vehicle**.
 - A **Motorcycle** **is a** **Vehicle**.
- Example:

```
class Vehicle {
public:
    void startEngine() {
        std::cout << "Vehicle engine started.\n";
    }
    void stopEngine() {
        std::cout << "Vehicle engine stopped.\n";
    }
    void accelerate() {
        std::cout << "Vehicle is moving.\n";
    }
    void brake() {
        std::cout << "Vehicle is slowing down.\n";
    }
};
```



```

class Car : public Vehicle { // Car inherits from Vehicle
public:
    void openDoors() {
        std::cout << "Car doors opened.\n";
    }
    void turnOnRadio() {
        std::cout << "Radio turned on.\n";
    }
    // Override the accelerate method for Car
    void accelerate() {
        std::cout << "Car is accelerating quickly!\n";
    }
};

class Motorcycle : public Vehicle { // Motorcycle inherits from Vehicle
public:
    void lean() {
        std::cout << "Motorcycle is leaning into the turn.\n";
    }
};

int main() {
    Car myCar;
    Motorcycle myMotorcycle;

    // Using inherited methods
    myCar.startEngine(); // Inherited from Vehicle
    myMotorcycle.stopEngine(); // Inherited from Vehicle
    std::cout << std::endl;
    // Using methods specific to the subclass
    myCar.openDoors();
    myMotorcycle.lean();
    std::cout << std::endl;
    // Using an overridden method
    myCar.accelerate(); // Calls the Car's accelerate method
    myMotorcycle.accelerate(); // Calls the Vehicle's accelerate method

    return 0;
}

/*
Vehicle engine started.
Vehicle engine stopped.

Car doors opened.
Motorcycle is leaning into the turn.

Car is accelerating quickly!
Vehicle is moving.
*/

```

- Some more examples:
 - **class:** `Animal` → **Derived:** `Dog, Cat, Bird`
 - **Base:** `eat(), sleep()`
 - **Derived** adds: `bark(), meow(), fly()`

```
class Animal {
public:
    void eat() {
        std::cout << "Animal is eating.\n";
    }
    void sleep() {
        std::cout << "Animal is sleeping.\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        std::cout << "Dog is barking.\n";
    }
};

class Cat : public Animal {
public:
    void meow() {
        std::cout << "Cat is meowing.\n";
    }
};

class Bird : public Animal {
public:
    void fly() {
        std::cout << "Bird is flying.\n";
    }
};
```

5. Combine Concepts

- Here are scenarios combining all 3 pillars:
 - Game Example
 - **Base:** `Character` → **Derived:** `Knight, Archer`
 - **Encapsulated:** `health, stamina`
 - **Abstracted:** `attack(), move()`
 - Library System
 - **Base:** `LibraryItem` → **Derived:** `Book, DVD, Magazine`
 - **Encapsulated:** `title, id, availability`
 - **Abstracted:** `checkout(), returnItem()`
 - Smart Devices
 - **Base:** `SmartDevice` → **Derived:** `SmartSpeaker, SmartLight`
 - **Encapsulated:** `deviceId, status`
 - **Abstracted:** `turnOn(), turnOff()`
 - Vehicle System
 - **Base:** `Vehicle` → **Derived:** `Car, Bike, Truck`
 - **Encapsulated:** `speed, fuelLevel`
 - **Abstracted:** `startEngine(), drive(), stopEngine()`
 - Hospital Management System
 - **Base:** `Staff` → **Derived:** `Doctor, Nurse, Technician`
 - **Encapsulated:** `name, staffID, shiftTimings`
 - **Abstracted:** `performDuty(), checkAvailability()`
 - E-commerce System
 - **Base:** `User` → **Derived:** `Buyer, Seller`
 - **Encapsulated:** `userID, address, paymentInfo`
 - **Abstracted:** `placeOrder(), addToCart(), updateProfile()`

6. Polymorphism

- To truly understand polymorphism, a beginner must first understand:
 1. References and pointers to base classes.
 2. Virtual functions.

References

- A **reference** is like a **nickname or alias** for something that already exists.
- It doesn't store the actual data itself — instead, it gives you a **way to access or manipulate** that data **indirectly**.

Real-Life Analogy



- Imagine:
 - You have a book (the actual data)
 - You give it a **sticky note** labelled "Favourite Book" — that note doesn't contain the book; it just refers to the book.



- Now, if someone says:
 - "Bring me the 'Favourite Book',"
 - They're using the label/reference, not the book itself — but they still get access to the actual book.

In Programming Terms

- In programming, a reference:
 - Is **not a copy** of the data.
 - It's a way to **access the original data** through another name.
- Example: Let's say we have a variable holding a number:

```
x = 5
ref = reference to x
```

- Now:
 - If we change **ref** to **10** → **x** also becomes **10**.
 - Because **ref** is just another way of accessing **x**.

Reference in Python

- Everything in Python is an object, and variables act as names or labels that point to these objects. This "pointing" is what we refer to as **references**.
- **Mutable Objects (like lists, dictionaries, sets):** When you modify a mutable object through one reference, all other references to that same object will see the change.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # Output: [1, 2, 3, 4]
```

- a and b reference the same list object.
 - There is no explicit syntax for references, but assignment always copies the reference (not the object).
 - Immutable types (like **int**, **str**, **tuple**) behave differently since you can't modify them in place.
- **Immutable Objects (like integers, floats, strings, tuples):** When you perform an operation that seems to modify an immutable object, Python actually creates a *new* object in memory, and the variable is then reassigned to refer to this new object. Other variables that were referencing the original immutable object remain unchanged.

```
x = 5
y = x
print(f"x: {x}, y: {y}") # Output: x: 5, y: 5

x = x + 1
print(f"x: {x}, y: {y}") # Output: x: 6, y: 5
```

Reference in Java

- In **Java**, all **non-primitive types** (like objects, arrays, etc.) are handled via **references**.

```
class Dog { String name; }
Dog a = new Dog();
Dog b = a;
b.name = "Buddy"; // a.name is also "Buddy"
```

- Primitive types (**int**, **char**, etc.) are passed **by value**, but objects are passed **by reference to the object**.

Reference in C#

- In **C#**, value types (like **int**, **struct**) are **copied**, and reference types (like **class**) are **passed by reference**.

```
class Dog { public string name; }  
Dog a = new Dog();  
Dog b = a;  
b.name = "Max"; // a.name is also "Max"
```

Reference in C++

- In **C++**, a **reference** is essentially an **alias** for another variable. It's declared using the **&** symbol:

```
int x = 10;  
int& ref = x;  
ref = 20; // x is now 20
```

- References must be initialized when declared.
- Once bound, a reference cannot be changed to refer to another variable.
- Useful for passing variables by reference to functions (to avoid copying and allow modifications).

Two Main Types of Polymorphism

1. Compile-time Polymorphism (Static Polymorphism or Method Overloading)
2. Run-time Polymorphism (Dynamic Polymorphism or Method Overriding)

1. Compile-time Polymorphism

- **Method overloading** means you can have multiple methods in the **same class** with the **same name** but **different parameter lists** (different number of parameters or different types).
- The compiler matches the method call to the most specific method signature it can find.
 - Number of arguments
 - Type of each argument
 - Order of arguments

```
// C++
class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
};

int main() {
    Calculator calc;
    calc.add(1, 2);           // calls int add(int, int)
    calc.add(1.1, 2.2);       // calls double add(double, double)
    calc.add(1, 2, 3);         // calls int add(int, int, int)
}
```

```
// Java
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

```
// C#
class Calculator {
    public int Add(int a, int b) => a + b;
    public double Add(double a, double b) => a + b;
    public int Add(int a, int b, int c) => a + b + c;
}
```

- Python (Doesn't Support Overloading in the Same Way)

```
class Calculator:
    def add(self, *args):
        return sum(args)

calc = Calculator()
print(calc.add(1, 2))           # 3
print(calc.add(1, 2, 3))       # 6
```

2. Run-time Polymorphism

- Let's first understand **function overriding**.
- The function in the **derived class** is said to override or hide the **base class** function because it has the **same name and signature**.

```
#include <iostream>
using namespace std;

class Animal {
public:
    void sound() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* a = new Dog();
    a->sound(); // Output: Animal sound (NOT Dog barks!)
    delete a;
}
```

- When a **base class** (`Animal* a`) pointer/reference points to a **derived class** object (`Dog`), and a function is called, it blindly calls the **base class version**.

Please understand the above statement thoroughly!

Virtual Functions

- A **virtual function** is a function that is **declared in a base class** and is **meant to be overridden in derived classes**.
- When a **base class** (`Animal* a`) pointer/reference points to a **derived class** object (`Dog`), and a **virtual** function is called, the **derived class version is executed**, not the base **class** version.
- This behaviour is called **dynamic dispatch** or **runtime polymorphism**.

```
#include <iostream>
using namespace std;

class Animal {
public:
    void virtual sound() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* a = new Dog();
    a->sound(); // Output: Dog barks
    delete a;
}
```

- In Java, **all non-static and non-final methods are implicitly virtual**. You don't need a special keyword like **virtual**.
- C# is similar to C++ in that you need to explicitly use the **virtual** keyword in the base **class** to make a method virtual. Derived **classes** then use the **override** keyword to provide their specific implementation.
- Python doesn't have explicit keywords like **virtual** or **override**. Python is a dynamically typed language, and method resolution (determining which method to call) happens at runtime based on the actual type of the object.

Summary: Dynamic Polymorphism

- **Dynamic polymorphism** involves multiple **classes** related by **inheritance** (a parent-child relationship).
- **Analogy:** If a **parent** is invited to an event, the **child** can also attend on the parent's behalf. This means a **derived class object** can be passed to a function that expects a **base class pointer or reference**.
- To enable dynamic polymorphism, we **mark the method as virtual in the base class** (in C++/C#). In Java and Python, methods are **virtual** by default.
- We use a **base class pointer or reference** to point to a **derived class object**, and when we call the method, the **derived class's overridden method is executed** at runtime.

7. Abstract Classes and Abstract Methods

- Abstract classes in **Java**, **C#**, and **C++** serve similar purposes.
- They define a base class that **cannot be instantiated**.
- It's designed to be inherited by other classes (subclasses) that provide the concrete implementation for its abstract members.
- Abstract classes **can contain**:
 - Abstract methods.
 - Concrete (fully implemented) methods
 - Instance variables.
- You declare an abstract class using the **abstract** keyword (in many languages like **Java and C#**).

Abstract Method:

- An abstract method is a method declared within an abstract class but without any implementation (no method body).
- It essentially defines a signature (name, parameters, return type) that *must* be implemented by any concrete subclass that inherits from the abstract class.
- Abstract methods also use the **abstract** keyword in **Java and C#**.

Examples:

- Java:

```
abstract class Animal {  
    abstract void makeSound(); // abstract method  
    void eat() {                // concrete method  
        System.out.println("This animal eats food.");  
    }  
}  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Woof");  
    }  
}
```

- All subclasses must implement all abstract methods unless they themselves are declared abstract.

- C#

```
abstract class Animal {
    public abstract void MakeSound();
    public void Eat() {
        Console.WriteLine("This animal eats food.");
    }
}

class Dog : Animal {
    public override void MakeSound() {
        Console.WriteLine("Woof");
    }
}
```

- C++

```
class Animal {
public:
    virtual void makeSound() = 0; // pure virtual function
    void eat() {
        std::cout << "This animal eats food." << std::endl;
    }
};

class Dog : public Animal {
public:
    void makeSound() override {
        std::cout << "Woof" << std::endl;
    }
};
```

Summary Comparison:

Feature	Java	C#	C++
Keyword	abstract	abstract	Pure virtual functions (= 0)
Abstract methods allowed?	Yes	Yes	Yes
Concrete methods allowed?	Yes	Yes	Yes
Can be instantiated?	No	No	No (if has pure virtual fn)

8. Interfaces

- An **interface** is a **contract** that defines **what** methods a class must implement, but **not how** they are implemented.
- Interfaces contain **only method signatures** (and possibly constants), and any class that implements the interface **must provide concrete implementations** of those methods.
- As we know...
 - Class is a blueprint for an entity
 - Interface is a blueprint for behaviour.

Examples:

- Java

```
interface PaymentMethod {  
    void processPayment(double amount);  
}  
  
class CreditCardPayment implements PaymentMethod {  
    public void processPayment(double amount) {  
        System.out.println("Processing credit card payment of $" + amount);  
    }  
}  
  
class PayPalPayment implements PaymentMethod {  
    public void processPayment(double amount) {  
        System.out.println("Processing PayPal payment of $" + amount);  
    }  
}
```

- Declared using the **interface** keyword.
- All methods are implicitly **public** and **abstract** (except default and static methods).
- Fields are implicitly **public static final** (constants).

- C#

```
interface IPaymentMethod {
    void ProcessPayment(double amount);
}

class CreditCardPayment : IPaymentMethod {
    public void ProcessPayment(double amount) {
        Console.WriteLine($"Processing credit card payment of ${amount}");
    }
}

class PayPalPayment : IPaymentMethod {
    public void ProcessPayment(double amount) {
        Console.WriteLine($"Processing PayPal payment of ${amount}");
    }
}
```

- Uses the **interface** keyword.
- All methods are public and cannot **have implementation**.
- Fields are not allowed.

- C++

```
class PaymentMethod {
public:
    virtual void processPayment(double amount) = 0;
    virtual ~PaymentMethod() = default;
};

class CreditCardPayment : public PaymentMethod {
public:
    void processPayment(double amount) override {
        cout << "Processing credit card payment of $" << amount << endl;
    }
};

class PayPalPayment : public PaymentMethod {
public:
    void processPayment(double amount) override {
        cout << "Processing PayPal payment of $" << amount << endl;
    }
};
```

- C++ **doesn't** have **interface keyword**.
- You create an interface by defining a class with only pure virtual functions.

9. When to use Abstract classes and when to use Interfaces

- Choosing between **abstract classes** and **interfaces** depends on your design needs, and while they sometimes overlap in function, they serve **different purposes**.

Use Abstract Classes When:

Situation	Reason
You want to share code (common logic) across related classes	Abstract classes can have both abstract methods and concrete (implemented) methods
You want to define base behaviour and allow overrides	You can provide default behaviour, and derived classes can override it selectively
You want to maintain a strong "is-a" relationship	Example: Dog is an Animal — both share common logic and identity
You need constructors, fields, or access modifiers	Abstract classes can have constructors, protected/private fields, and control access to members
You want to prevent multiple inheritance of implementation	Since only one abstract base class can be inherited, this avoids complexity

Use Interfaces When:

Situation	Reason
You want to define a contract or capability	Interfaces define <i>what</i> a class can do (like a plug-in), not <i>how</i> it does it
You need to support multiple inheritance	A class can implement multiple interfaces, but can inherit from only one class
You want to decouple components	Interfaces make it easy to swap out implementations without changing client code
You want to ensure loose coupling and testability	Especially useful for dependency injection and mocking in unit tests
You are working with plugin architectures or cross-cutting concerns	Like logging, serialization, or persistence behaviour

Rule of Thumb:

- Use an abstract class when you want to provide some default implementation and enforce a base identity.
- Use an interface when you want to define capabilities or behaviours that can be shared across unrelated classes.

Summary Comparison Table

Feature	Abstract Class	Interface
Can contain implementation	Yes	No
Constructors	Yes	No
Fields	Yes (non-final)	Constants only
Multiple inheritance	No (only one abstract class)	Yes
Access modifiers	Yes (public/protected/private)	public only methods
Use for shared logic?	Yes	Not recommended
Use for contracts only?	Possible but not ideal	Best choice

More example

- Abstract Class → "**is-a**" relationship
 - When you use an **abstract class**, you're saying that all subclasses are a *type* of the abstract class. It defines a **core identity or category** with **common structure and behaviour**.

```
abstract class Vehicle {  
    void startEngine() { System.out.println("Engine started"); }  
    abstract void drive();  
}  
  
class Car extends Vehicle {  
    void drive() { System.out.println("Driving a car"); }  
}
```

- Here, Car **is a** Vehicle. Every vehicle has an engine and can drive(), but the driving method may vary. Shared behaviour like startEngine() is inherited.
- Interface → "**can-do**" capability
 - An **interface** defines a **capability or behaviour** that a class *can perform*, regardless of its position in a class hierarchy. It doesn't describe what a thing *is*, but rather what it *can do*.


```

interface Flyable {
    void fly();
}

class Bird implements Flyable {
    public void fly() { System.out.println("Bird flies"); }
}

class Airplane implements Flyable {
    public void fly() { System.out.println("Airplane flies"); }
}

```

- Bird and Airplane are not related, but both can fly. The interface lets them be treated polymorphically where flying capability is needed, without forcing them into the same class tree.
- Combining Both

```

abstract class Employee {
    abstract double calculateSalary();
}

interface Reportable {
    void generateReport();
}

class Manager extends Employee implements Reportable {
    double calculateSalary() { return 5000.0; }
    public void generateReport() {
        System.out.println("Manager report");
    }
}

```

- Manager **is an** Employee → inherits structure and core methods.
- Manager **can** generate reports → provides capability via Reportable.

Abstract Class (is-a)	Interface (can-do)
Defines <i>identity</i>	Defines <i>capability</i>
Inheritance relationship	Contract for behaviour
Shared logic and structure	Multiple behaviours for unrelated classes
Single inheritance	Multiple interfaces